ОТЧЁТНАЯ ДОКУМЕНТАЦИЯ

РУКОВОДСТВО РАЗРАБОТЧИКА

Информационная система «КнигоУчёт»

Веб-приложение для учёта библиотечного фонда

Сведения	**Заполнить перед сдачей**
Выполнили	Студенты группы ИСП 211
ФИО	Абалканов Чынгыз Тологонович Волков Владислав Валерьевич
Учебная дисциплина	Внедрение и сопровождение программного обеспечения

2026

1. Общие сведения и технологический стек

КнигоУчёт — учебное серверное веб-приложение для учёта библиотечного фонда. Архив содержит проект Django, шаблоны интерфейса, стили, REST API, тесты и команду заполнения демонстрационными данными.

Компонент	Использование
Python 3.12+	Язык приложения; поддержка также зависит от версии Django
Django 5.2 LTS	Маршрутизация, сессии, формы, административный интерфейс и доступ к данным
Django REST Framework 3.18	JSON API, сериализация, фильтрация и пагинация
Simple JWT	Выдача и обновление токенов для API
SQLite	База данных по умолчанию, не требует отдельной службы
PostgreSQL	Альтернативная база данных для развёртывания/учебных требований
HTML/CSS	Пользовательский интерфейс и адаптивные стили

Функциональные разделы



Рисунок 1 – Разделение функциональности

Названия и маршруты относятся к предметной области библиотеки: книги, читатели, выдачи и отчётность.

2. Структура исходного кода

```
book_ledger_project/
├── manage.py
├── requirements.txt
├── .env.example
├── start_windows.bat
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── apps/
│   ├── accounts/     # сотрудники, вход, seed_demo
│   ├── catalog/      # каталог и API книг
│   ├── readers/      # читательские карточки
│   ├── circulation/  # выдача, возврат, сервис и тесты
│   └── reports/      # KPI и CSV
├── templates/        # HTML-страницы
└── static/css/       # стили интерфейса
```

Назначение основных файлов

Файл	**Что содержит**
config/settings.py	Установленные приложения, база, язык, сессии, JWT и DRF
config/urls.py	Маршруты браузера, административного раздела, API и JWT
apps/*/models.py	Определения таблиц и связи для ORM Django
apps/*/forms.py	Проверка вводимых пользователем данных
apps/*/serializers.py	Валидация JSON-запросов и формирование JSON-ответов
apps/*/views.py	Обработчики браузерных страниц
apps/*/api.py	API-представления и CRUD-операции
apps/circulation/services.py	Транзакционная логика выдачи и возврата
apps/reports/services.py	Сводные показатели фонда

3. Структура данных и связи

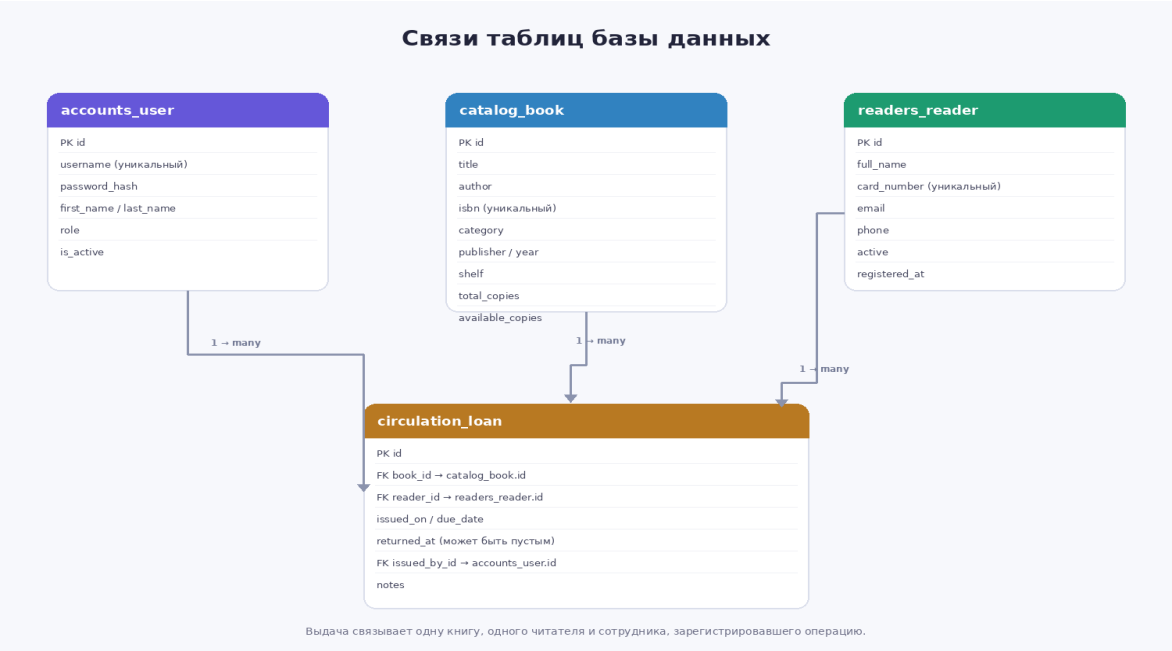


Рисунок 2 – ER-диаграмма базы данных

Таблица	**Связь / правило**
accounts_user	Одна учётная запись может быть ответственным сотрудником для многих выдач
catalog_book	Одна книга может присутствовать во многих записях выдачи
readers_reader	Один читатель может иметь историю выдач
circulation_loan	Связывает книгу, читателя и сотрудника; returned_at завершает выдачу

Правила учёта

Общее количество total_copies — число экземпляров в фонде. Доступное количество available_copies меняется только при выдаче и возврате. В форме редактирования общее число нельзя уменьшить ниже уже выданного количества. ISBN и номер читательского билета уникальны; ISBN необязателен.

Связи на книге и читателе используют защиту от удаления, чтобы исторические операции не терялись. Возврат устанавливает дату returned_at. Статус вычисляется динамически: returned — возвращена, overdue — срок прошёл и возврата нет, issued — находится на руках.

4. Установка и первый запуск

Windows: рекомендуемый порядок

1. Установить Python 3.12 или более новую поддерживаемую версию. При установке включить Python Launcher и PATH.
2. Распаковать архив в путь без специальных символов, например C:\study\book_ledger.
3. Открыть терминал в корневой папке проекта.
4. Создать venv: `py -m venv .venv`.
5. Активировать: `.venv\Scripts\activate`.
6. Установить библиотеки: `python -m pip install -r requirements.txt`.
7. Создать начальные миграции: `python manage.py makemigrations accounts catalog readers circulation`.
8. Применить миграции: `python manage.py migrate`.
9. Создать демонстрационные данные: `python manage.py seed_demo`.
10. Запустить: `python manage.py runserver 127.0.0.1:8000`.

Автоматизированный запуск

На Windows предусмотрен start_windows.bat. Скрипт создаёт виртуальное окружение, устанавливает зависимости, формирует и применяет миграции, наполняет базу демонстрационными данными и запускает сервер. Требуется подключение к интернету на этапе установки Python-пакетов.

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python manage.py makemigrations accounts catalog readers circulation
python manage.py migrate
python manage.py seed_demo
python manage.py runserver 127.0.0.1:8000
```

5. Конфигурация базы данных и миграции

SQLite — режим по умолчанию

Без дополнительных настроек приложение использует файл book_ledger.sqlite3 в корне проекта. Этот вариант предназначен для первого локального запуска, разработки и учебной демонстрации.

Подключение PostgreSQL

11. Установите PostgreSQL и создайте базу book_ledger с кодировкой UTF-8.
12. Скопируйте .env.example в .env.
13. Установите DB_ENGINE=postgresql, затем укажите DB_NAME, DB_USER, DB_PASSWORD, DB_HOST и DB_PORT.
14. Проверьте, что используется драйвер psycopg из requirements.txt и служба базы данных запущена.
15. Выполните `python manage.py migrate` и заполните нужные данные.

Миграции при изменении структуры

После добавления/изменения полей сформируйте миграцию `python manage.py makemigrations`, изучите созданный файл и примените её `python manage.py migrate`. В командной работе миграции следует включать в систему контроля версий. Не редактируйте применённые миграции задним числом.

Демонстрационные данные

Команда `python manage.py seed_demo` добавляет набор книг и читателей и создаёт демонстрационные выдачи, если журнал пуст. Учётная запись admin с паролем admin12345 создаётся только при отсутствии пользователя. Пароль необходимо сменить до работы с реальными данными.

6. API и аутентификация

Метод	Маршрут	Назначение
GET, POST	/api/books/	Список, создание книги
GET, PUT, PATCH, DELETE	/api/books/{id}/	Получение, изменение, удаление книги
GET, POST	/api/readers/	Список и создание читателей
GET, PUT, PATCH, DELETE	/api/readers/{id}/	Карточка читателя
GET, POST	/api/loans/	Список выдач и новая выдача
POST	/api/loans/{id}/return/	Отметить возврат
GET	/api/reports/kpi/	Сводные показатели
POST	/api/auth/token/	Получение пары access/refresh
POST	/api/auth/token/refresh/	Обновление access-токена

Получение JWT

```
curl -X POST http://127.0.0.1:8000/api/auth/token/ \
  -H "Content-Type: application/json" \
  -d "{\"username\":\"admin\",\"password\":\"admin12345\"}"
```

В ответе приходит JSON с access и refresh. Защищённые запросы передают `Authorization: Bearer <access>`. Время жизни access-токена задано в настройках и равно 30 минутам; refresh-токен по умолчанию действует один день. После истечения access используется маршрут refresh.

Пример создания выдачи

```
{
  "book": 1,
  "reader": 1,
  "due_date": "2026-11-06",
  "notes": "API example"
}
```

Внутренний сервис выдачи проверяет статус читателя, наличие экземпляра и дату возврата, затем создаёт операцию и уменьшает остаток в транзакции. При ошибке клиент получает сообщение в ответе API.

7. Бизнес-правила и проверка качества

Сервис выдачи

`apps/circulation/services.py` содержит функцию issue_book. Она выполняется в транзакции, читает запись книги с блокировкой, проверяет active у читателя, available_copies и дату due_date, создаёт журнал операции, а затем уменьшает доступный остаток. В случае ошибки транзакция откатывается.

Сервис возврата

Функция return_book проверяет, что запись существует и ещё не возвращена. Она фиксирует локальную дату возврата и увеличивает доступный остаток. Повторный возврат отклоняется. Статус просрочки вычисляется по due_date относительно текущей локальной даты, при этом завершённая операция не считается просроченной.

Ручные интеграционные проверки

- Проверить вход и закрытые страницы.
- Добавить книгу и найти её в каталоге.
- Выдать книгу, затем оформить возврат.
- Проверить просрочку и показатели обзора.
- Получить JWT и проверить API.
- Экспортировать CSV и проверить файл.

8. Развёртывание, безопасность и устранение ошибок

Ограничения локального сервера

`python manage.py runserver` подходит для разработки и демонстрации на localhost. Для публичного размещения нужно применить рекомендованный Django способ развёртывания через WSGI/ASGI-сервер и прокси, настроить HTTPS, разрешённые домены, статические файлы, журналирование и резервные копии.

Минимальные правила безопасности

- Не передавайте файл .env и не храните SECRET_KEY в репозитории.
- Установите DEBUG=0 при публичном развёртывании и заполните ALLOWED_HOSTS.
- Смените демо-пароль; создавайте отдельные учётные записи сотрудникам.
- Не публикуйте скриншоты с реальными данными читателей или живыми токенами.
- Храните резервные копии базы вне рабочей папки и периодически проверяйте восстановление.

Частые проблемы

Сообщение	**Решение**
ModuleNotFoundError: django	Активировать виртуальное окружение и установить requirements.txt
No such table / relation does not exist	Выполнить `python manage.py migrate`; при необходимости `makemigrations`
401 Unauthorized / 403 Forbidden	Получить токен, проверить заголовок Authorization или войти через форму
Port is already in use	Остановить предыдущий сервер или выбрать другой порт
PostgreSQL connection refused	Проверить службу, host/port, имя БД и пользователя
ISBN already exists	Использовать уникальный ISBN или оставить его пустым, если код неизвестен

9. Сопровождение и контрольный список

Перед демонстрацией

- Активировать окружение и выполнить python manage.py check.
- Применить миграции и запустить seed_demo на учебной базе.
- Выполнить python manage.py test apps.circulation и устранить ошибки.
- Открыть обзор, каталог, читателей, выдачи, отчёты, /api/ и /admin/.
- Сделать реальные снимки страниц работающего приложения для отчёта.
- Изменить пароль демонстрационного администратора, если база будет сохранена.

Перспективы развития

Возможные улучшения: QR-коды для читательских билетов, история изменений, расширенный поиск, фильтры отчётов по датам, импорт каталога из CSV, уведомления о скором сроке возврата, Docker и автоматизированный CI. Каждое изменение следует сопровождать миграцией, тестом и обновлением пользовательской документации.

Источники

- Django: https://docs.djangoproject.com/
- Django REST framework: https://www.django-rest-framework.org/
- Simple JWT: https://django-rest-framework-simplejwt.readthedocs.io/
- PostgreSQL: https://www.postgresql.org/docs/

Версии зависимостей зафиксированы в requirements.txt. Проверяйте официальные release notes перед обновлением.